ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Faculty of Information Technology

Course: Web Technology

Academic Year: 2026 - 2027

Instructor: Jeremie U. Tuyisenge

FINAL ACADEMIC DOCUMENTATION

SUPERmarket Inventory & Sales Manager

A centralized web application for supermarket products, inventory, sales, suppliers, branches, stock movements, alerts, reports and authorization

Student: IZA KURADUSENGE Emma Lise

Student ID: 28246

Repository: https://github.com/Emmalise1/Supermarket-Inventory-Sales-Manager

Main branch - pull request submitted on completion

October 2026

Contents

1. Problem statement
2. Domain and data modelling
3. Frontend, UI and UX
4. Backend architecture
5. Databases (MySQL and MongoDB)
6. Security, authentication and authorization
7. RabbitMQ messaging
8. Redis caching (additional technology)
9. Testing and quality assurance
10. Git and GitHub workflow
11. DevOps and continuous integration
12. Limitations and unimplemented items
Appendix A - Demo accounts and running the system
Appendix B - Coverage of the assignment requirements

List of figures

Figure 1: UML use case diagram
Figure 2: UML class diagram of the domain model
Figure 3: UML state machine diagram - life cycle of a Product
Figure 4: Responsive wireframes (desktop, tablet, mobile)
Figure 5: UML deployment diagram
Figure 6: UML sequence diagram - recording a POS sale
Figure 7: UML sequence diagram - RabbitMQ event flow
Figure 8: UML activity diagram (swimlanes) - barcode lookup with Redis
Figure 9: UML activity diagram (swimlanes) - write path and cache invalidation

1. Problem statement

1.1 Real-world problem

Supermarkets and small retail shops in the region usually manage products, stock and sales with paper record books and scattered spreadsheets. Each branch keeps its own files, so stock levels are duplicated, outdated or simply lost. When a product runs out, nobody notices until a customer asks for it. When a sale is recorded twice or a price is typed incorrectly, the mistake is found only at stock-counting time. Basic management questions also have no reliable answer: which products sell best, which branch is profitable, and who changed a stock level and why.

The consequences are wasted time, stock-outs, overstock, untraceable losses and no usable history. This project therefore builds one centralized web application that keeps the products, stock movements and sales of every branch in a single system, with controlled access, automatic low-stock alerts and reports computed from real data instead of manual consolidation.

1.2 Target users

Actor	Profile	Main needs
Administrator	Shop owner or IT officer with access to all branches	Create users and branches, full control, audit history, reports
Store manager	Responsible for one branch	Receive goods, adjust stock, manage products, watch low stock, run reports
Cashier	Sells at the point of sale of one branch	Fast barcode lookup, POS checkout, stock alerts, own notifications

1.3 Objectives

General objective

To design and implement a centralized web application that replaces paper records and scattered spreadsheets with a single, secure and reliable system for supermarket inventory and sales management.

Specific objectives

- Manage products, categories, suppliers, branches and user accounts from one interface.
- Record every stock change (receiving, sale, adjustment) in an immutable movement history.
- Prevent invalid states such as negative stock, duplicate barcodes or unrecorded sales.
- Restrict each user to the functions and the branch of their role (RBAC plus branch scoping).
- Answer barcode lookups and dashboard queries quickly through a Redis cache layer.
- Notify the right people asynchronously about sales, goods receiving and low stock through RabbitMQ.
- Keep a durable audit trail of who did what, stored in a document database (MongoDB).
- Automate build and test verification with a CI pipeline (GitHub Actions).

1.4 Scope

In scope

- Web application (React single-page app) and REST backend (Spring Boot), accessed through a browser.
- Inventory per branch: products, goods receiving, stock adjustments, movement history and low-stock alerts.
- POS sales with stock deduction, receipt numbers and price snapshots.

- Reference data: categories, suppliers, branches and users.
- Dashboard, sales reports, audit history and notifications.
- Authentication (JWT), role-based and branch-level authorization.
- MySQL (operational data), MongoDB (audit and notifications), Redis (cache) and RabbitMQ (events).

Out of scope

- Hardware integration (physical barcode scanners, receipt printers): the barcode is typed or pasted.
- Online payment, invoicing, accounting and payroll.
- Native mobile applications: the user interface is responsive instead.
- Docker and containers: the system runs on plain local services, as required for this project.
- Multi-tenant or internationalization features.

1.5 Functional and business requirements

ID	Requirement	Type	Implemented in
FR-01	Log in with email and password and receive a signed access token	Functional	POST /api/auth/login
FR-02	Create, update and deactivate products with a unique barcode	Functional	ProductService, /api/products
FR-03	Look up a product by barcode for POS scanning	Functional	GET /api/products/barcode/{barcode}
FR-04	Record a POS sale: merge lines, price from database, total, receipt number	Functional	POST /api/sales
FR-05	Receive supplier goods and increase stock with a movement record	Functional	POST /api/inventory/goods-receiving
FR-06	Adjust stock manually with a mandatory reason	Functional	POST /api/inventory/adjustments
FR-07	Show low-stock products against their thresholds	Functional	GET /api/products/low-stock
FR-08	Notify branch users about sales, receipts and low stock	Functional	RabbitMQ -> /api/notifications
FR-09	Record an audit entry for every important change	Functional	AuditService -> MongoDB audit_logs
FR-10	Produce sales reports (revenue, count, average, top products)	Functional	GET /api/reports/sales
FR-11	Manage users, roles and branch assignment	Functional	GET/POST/PUT /api/users (ADMIN)
BR-01	Stock can never become negative; the operation is rejected instead	Business	SaleService, InventoryService
BR-02	Sale prices always come from MySQL, never from the client request	Business	SaleService.create
BR-03	Every stock change writes a StockMovement row in the same transaction	Business	@Transactional services
BR-04	A non-administrator only accesses data of their own branch	Business	AuthContext.requireBranchAccess
BR-05	Barcodes and sale numbers are unique	Business	Unique index + service check
BR-06	A user cannot deactivate their own account	Business	UserService.update
NFR-01	Barcode lookup is served from Redis when possible (cache hit reported to the client)	Quality	ProductService + ProductCacheService
NFR-02	A Redis or RabbitMQ outage must not break the request path	Quality	Fallbacks in cache and publisher code
NFR-03	A health check is available for monitoring and CI	Quality	GET /actuator/health

1.6 Measurable quality attributes

Attribute	Target	How it is measured / verified
Usability	One responsive layout usable on desktop, tablet and phone; role-aware navigation	CSS breakpoint at 980 px; the sidebar becomes a top bar; routes are hidden by role (ProtectedRoute tests)
Performance	Barcode lookup served from cache without hitting MySQL; measurable per request	The API response carries cacheHit and elapsedMs; a benchmark endpoint compares both paths on the live system
Reliability	No request fails because Redis or RabbitMQ is down	Every Redis call and every publish is wrapped in a fallback; unit tests cover broker-down behaviour
Security	No unprotected endpoint, no plaintext password, no cross-branch data leak	Spring Security filter chain and @PreAuthorize; tests assert 401/403 for anonymous, forged and wrong-role tokens
Data integrity	No negative stock, no duplicate barcode, movements always consistent	Service-level validation inside transactions; H2 integration test on the unique barcode constraint
Maintainability	Automated verification on every push and pull request	GitHub Actions builds backend and frontend and runs all tests

1.7 User stories and acceptance criteria

User story 1 - Cashier at the point of sale

As a cashier of the Kigali Main branch, I want to scan or type a barcode and add the product to the current sale, so that I can serve customers quickly without typing prices.

- **AC1.1** Given I am logged in as CASHIER, when I request a barcode that exists, then the product is returned with its current price and stock, and the response states whether Redis or MySQL answered.
- **AC1.2** Given the requested quantity exceeds the available stock, when I submit the sale, then the sale is rejected with a clear message and no stock is changed.
- **AC1.3** Given a successful sale, then a unique sale number is created, stock is reduced, a stock movement of type SALE is written and the affected cache entries are invalidated.
- **AC1.4** Given I am not authenticated, then every product endpoint returns 401.

User story 2 - Store manager receiving goods

As a store manager, I want to receive a supplier delivery into my branch, so that stock levels reflect what physically arrived and my team is warned when something is running low.

- **AC2.1** Given a receipt of N > 0 units, when it is saved, then stock increases by N, a GOODS_RECEIPT movement is recorded with the user and timestamp, and the product cache is invalidated.
- **AC2.2** Given N is zero or negative, then the request is rejected with a business-rule error.
- **AC2.3** Given the new stock is at or below the low-stock threshold, then a LOW_STOCK event is published to RabbitMQ and a notification appears for the branch users.
- **AC2.4** Given a product of another branch, when a non-admin tries to receive it, then 403 is returned.

User story 3 - Administrator reviewing the history

As an administrator, I want to see who changed what and when, so that losses and mistakes can be traced back to a person and a moment instead of being guessed.

- **AC3.1** Given a product update, a sale or a login, then an audit document is written to MongoDB with actor, role, branch, action, entity and timestamp.
- **AC3.2** When I open the audit page, the latest 50 entries are listed, scoped to a branch for non-administrators.
- **AC3.3** A failed audit write never breaks the business operation that triggered it.

2. Domain and data modelling

2.1 Actors, processes and data objects

This section summarizes the principal domain concepts at an abstract level, independently of any technology choice, before implementation detail is introduced.

Actors. Three actors use the system: an administrator who owns the whole shop network, a store manager responsible for one branch, and a cashier who serves customers in one branch. An external supplier is not a user of the system but takes part in the domain by delivering goods to a branch.

Processes. Opening a session, maintaining the product catalogue, receiving goods from a supplier, correcting a recorded stock level, completing a point-of-sale transaction, monitoring stock levels and producing alerts, notifying the interested parties, analysing sales through reports, and reviewing the history of changes.

Data objects. The shop network (branches), the people entitled to use the system (users and their roles), the catalogue (products, categories, suppliers), the commercial records (sales and their lines), the inventory history (stock movements), and the information derived for the users (notifications and audit records).

Rules of the domain (invariants). A product belongs to exactly one branch and its quantity can never drop below zero. Every stock change is attributable to an actor and justified. A sale freezes the price of each line at the moment of the sale. Each actor sees the data of their own branch only, except the administrator.

Figure 1 models these actors and processes as a UML use case diagram. The three actors are related by generalization: a Store Manager can do everything a Cashier can, and an Administrator everything a Store Manager can, which mirrors the role matrix of section 6.4. Recording a POS sale always includes looking up the product by barcode, and every stock-changing use case (sale, goods receiving, adjustment) includes recording a stock movement. Sending a low-stock alert is an optional extension of that movement, taken only when the resulting quantity is at or below the low-stock threshold. Section 2.2 translates the concepts into classes and section 2.4 into the chosen persistence technology.

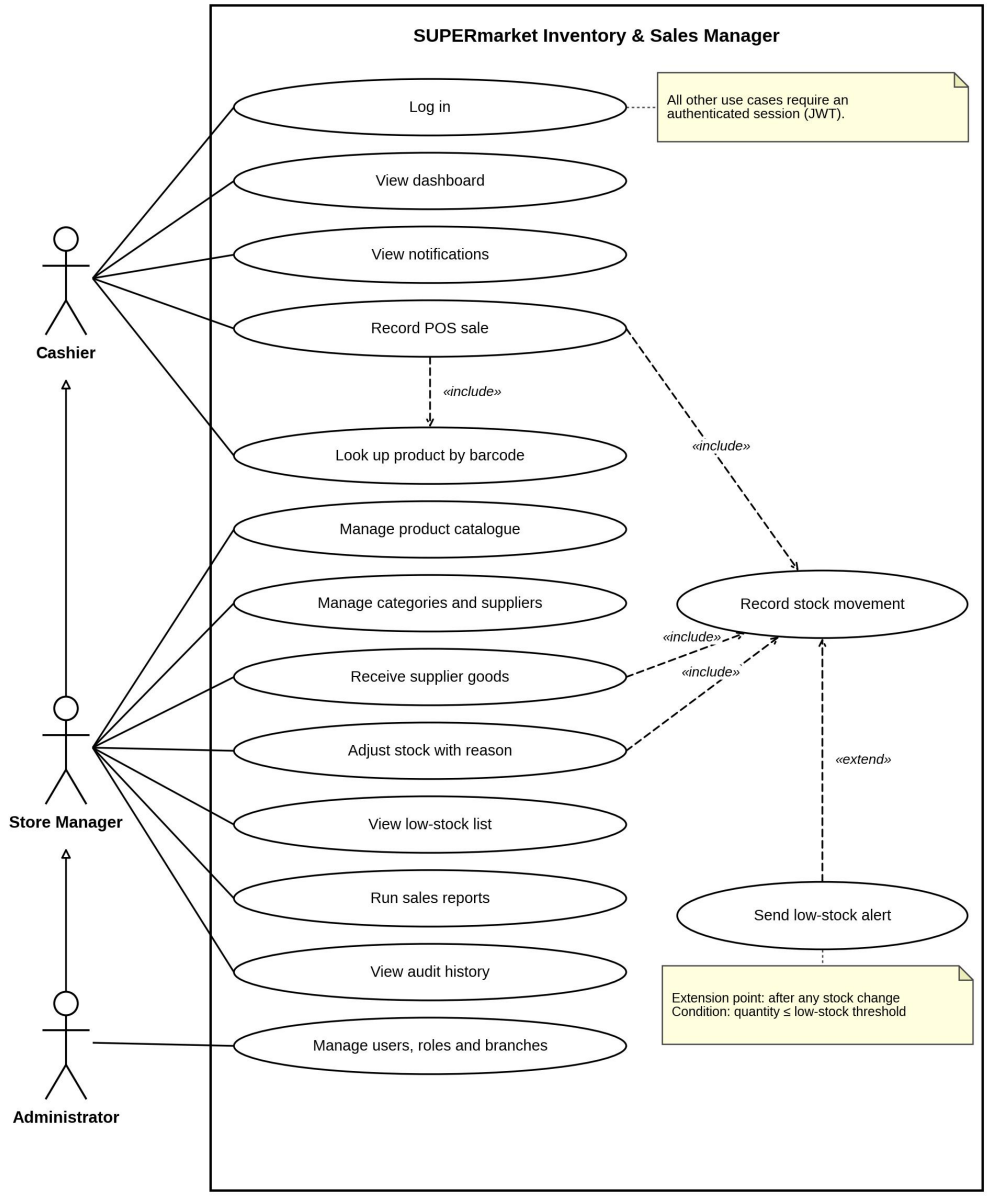


Figure 1: UML use case diagram - actors, system boundary, «include» and «extend» relationships of the system.

2.2 Class diagram

The UML class diagram in Figure 2 reflects the classes that actually exist in the source code (package com.supermarket.domain): eight persistent entity classes plus two enumerations (Role and MovementType). Each class shows its attributes as visibility name : Type and its operations. Associations carry multiplicities. The Sale-SaleItem relationship is a composition, because a SaleItem cannot exist without its Sale, and the dependencies on the enumerations are dashed. Category and Supplier are optional for a product (multiplicity 0..1). Getters and setters are omitted for readability. The table-level view of the same model, with keys and indexes, is given in section 2.4.

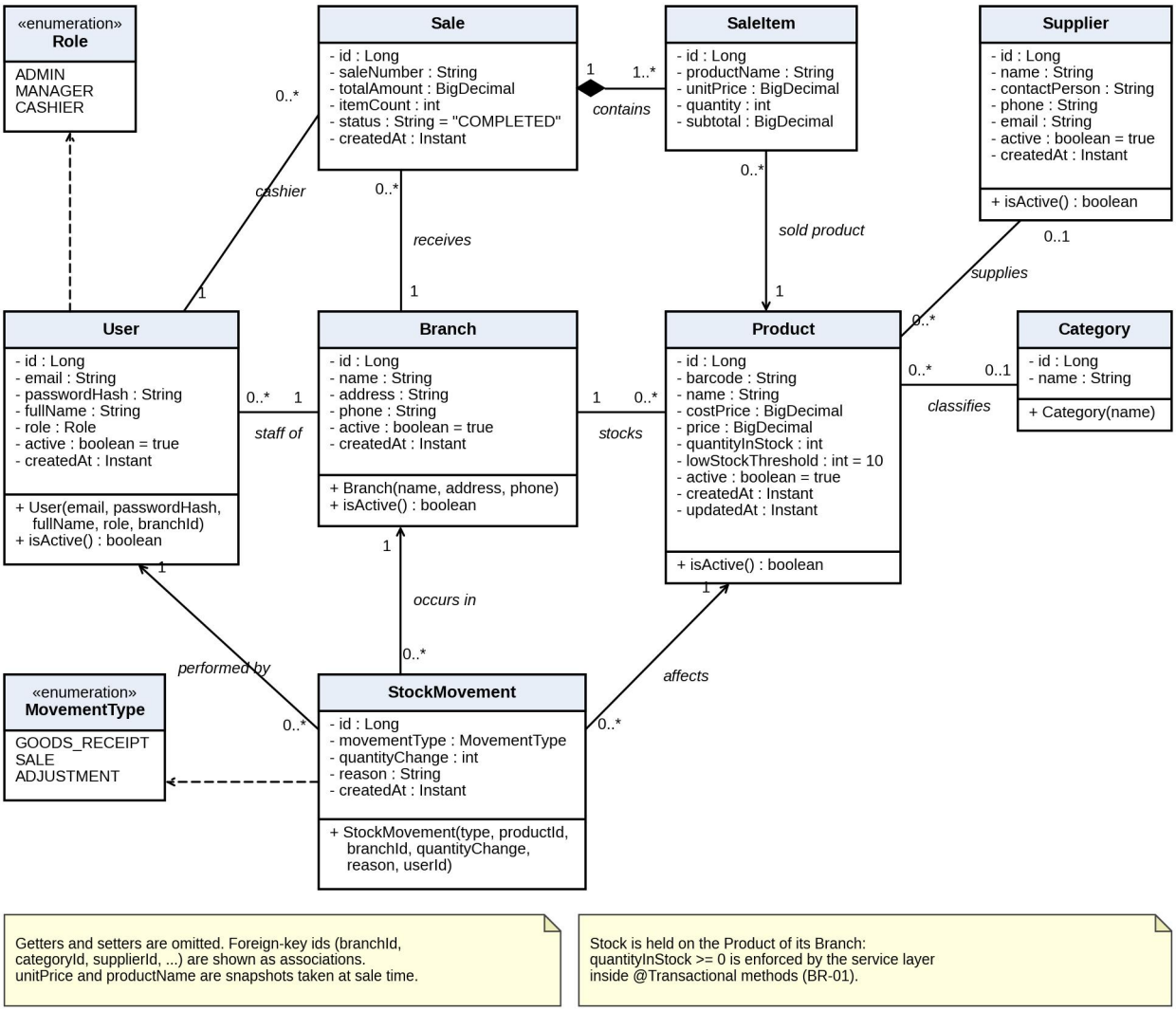


Figure 2: UML class diagram of the domain model (package com.supermarket.domain).

2.3 Core entities, attributes and relationships

Entity	Key attributes	Relationships
User	id (PK), email (unique), password_hash, full_name, role, branch_id (FK), active, created_at	Belongs to a Branch; authors Sales and StockMovements
Branch	id (PK), name (unique), address, phone, active, created_at	Owns users, products, sales and movements
Category	id (PK), name (unique)	Classifies products
Supplier	id (PK), name, contact_person, phone, email, active	Supplies products
Product	id (PK), barcode (unique), name, category_id (FK), supplier_id (FK), branch_id (FK), cost_price, price, quantity_in_stock, low_stock_threshold, active	Stock is stored directly on the product row of its branch
Sale	id (PK), sale_number (unique), branch_id (FK), cashier_id (FK), total_amount, item_count, status	Header of a sale; contains sale items
SaleItem	id (PK), sale_id (FK), product_id, product_name, unit_price, quantity, subtotal	Line of a sale with a price snapshot
StockMovement	id (PK), movement_type, product_id (FK), branch_id (FK), quantity_change, reason, user_id (FK)	Immutable history of every stock change

Design note

The system does not use separate BranchStock, GoodsReceipt or GoodsReceiptItem tables. Stock is a column of the product that already belongs to a branch (products.branch_id and products.quantity_in_stock), and a goods receipt is an operation that increases that column and writes a StockMovement of type GOODS_RECEIPT. This keeps the invariant in one place and makes the rule "one product = one branch stock line" impossible to violate.

2.4 Technology-oriented schema (MySQL)

Primary keys are BIGINT identity columns. A real foreign key exists where a join is essential: sale_items.sale_id references sales.id. The other references (products.category_id, products.supplier_id, products.branch_id, users.branch_id, sales.cashier_id, stock_movements.product_id and user_id) are stored as ID columns and enforced at application level. Tables and indexes are generated by JPA (ddl-auto: update) for local development.

Table	Indexes	Constraints
users	unique index on email	email unique NOT NULL; password_hash NOT NULL; role NOT NULL
branches	unique index on name	name unique NOT NULL
categories	unique index on name	name unique NOT NULL
suppliers	-	name NOT NULL
products	idx_products_barcode (UNIQUE), idx_products_branch, idx_products_name	barcode unique NOT NULL; price NOT NULL (>= 0 by Bean Validation); quantity_in_stock >= 0 by service rule
sales	idx_sales_branch, idx_sales_created	sale_number unique NOT NULL; total_amount NOT NULL
sale_items	PK id, FK sale_id -> sales	unit_price, quantity, subtotal NOT NULL
stock_movements	idx_movements_product, idx_movements_branch	movement_type NOT NULL; quantity_change signed NOT NULL

Business invariants are enforced in the service layer inside @Transactional methods, because Hibernate is configured with ddl-auto: update and does not generate CHECK constraints for them. Negative stock, zero adjustments, missing adjustment reasons and deactivated products are all rejected before any write happens.

2.5 MongoDB documents

Database supermarket, collection audit_logs

Field	Type	Index	Meaning
_id	ObjectId	primary	Document identifier
actorId	Number	indexed	User who performed the action
actorEmail / role / branchId	String / String / Number	branchId indexed	Context of the actor
action	String	-	USER_LOGIN, PRODUCT_UPDATED, SALE_COMPLETED, GOODS_RECEIVED, STOCK_ADJUSTED ...
entityType / entityId	String	-	Object affected, for example Product / 7
details	String	-	Human-readable description
createdAt	Instant	indexed	When it happened

Database supermarket, collection notifications

Field	Type	Index	Meaning
_id	ObjectId	primary	Document identifier
userId	Number	indexed	Recipient; null means a branch-wide notification
branchId	Number	indexed	Branch concerned
type	String	-	LOW_STOCK, SALE_COMPLETED, GOODS_RECEIVED, STOCK_ADJUSTED
title / message	String	-	Displayed content
read	Boolean	-	Read flag for the badge counter
createdAt	Instant	indexed	Creation time

2.6 Product stock life cycle (state machine)

A product is the object whose behaviour changes most with its state, so its life cycle is modelled as the UML state machine in Figure 3. When a product is created, a choice node places it in one of three stock states according to its quantity: InStock (above the low-stock threshold), LowStock (at or below the threshold) or OutOfStock (zero). Sales and negative adjustments move it downwards; goods receipts and positive adjustments move it upwards. Whenever a stock change leaves the quantity at or below the threshold, a LOW_STOCK event is published.

Deactivating a product moves the whole composite state Active to Inactive, where sales, receipts and adjustments are rejected. An update that sets active to true returns the product to the stock state it had before, which is expressed with a deep-history pseudostate. Events that would make the quantity negative raise a BusinessRuleException and leave the state unchanged (BR-01).

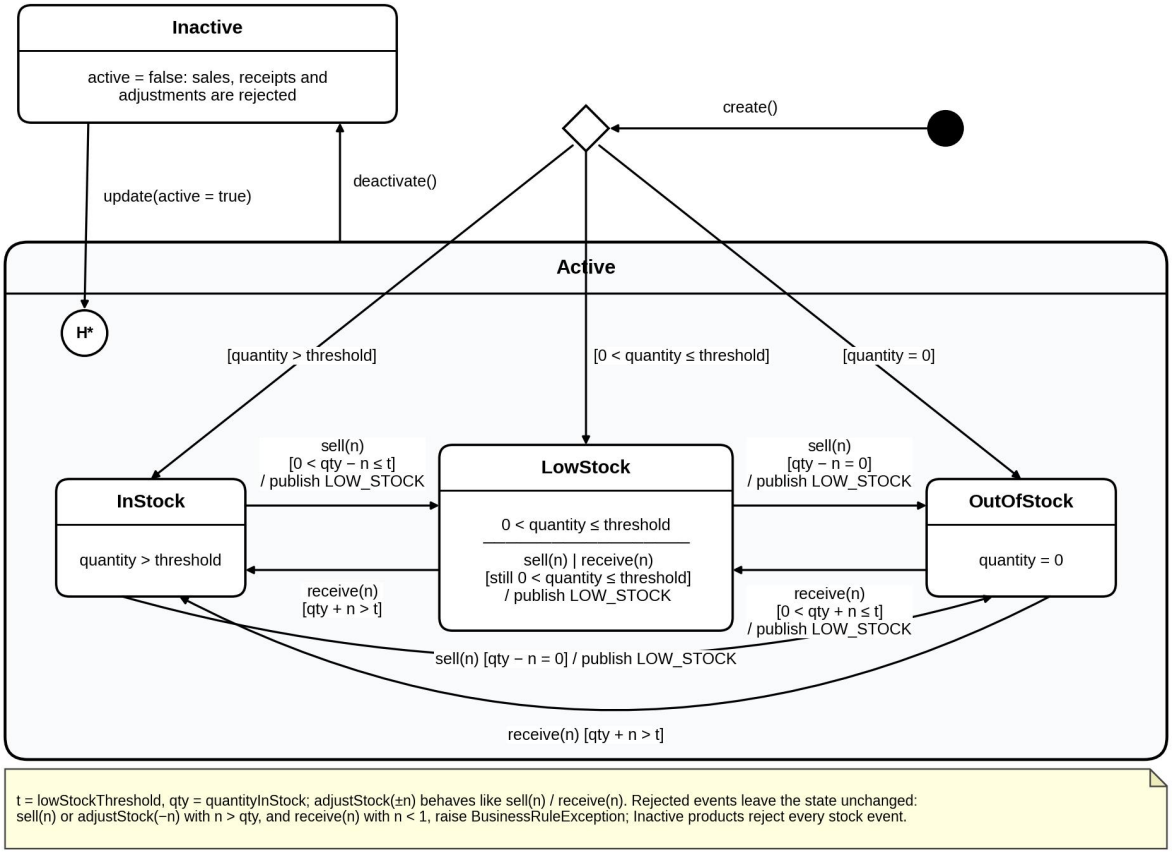


Figure 3: UML state machine diagram - life cycle of a Product (stock levels and active flag).

3. Frontend, UI and UX

3.1 Technology

- **Library:** React 19 with functional components and hooks.
- **Build tool:** Vite 6 (development server on port 5173, proxying /api to the backend on port 8080).
- **Routing:** React Router 7 with nested routes and role-guarded areas.
- **HTTP:** an Axios client with a request interceptor that attaches the JWT from localStorage and a response interceptor that logs the user out on a 401.
- **Styling:** hand-written responsive CSS (styles.css) with no UI framework, so the layout is fully controlled.
- **State:** a small AuthContext stores the current user and role; pages keep local state.

3.2 Pages and role-aware navigation

Route	Page	Access
/login	Login (public)	Anonymous
/dashboard	Dashboard summary	Any authenticated user
/pos	POS / Sales	Any authenticated user (branch-scoped)
/products	Products, barcode lookup, benchmark	Any authenticated user; write actions need ADMIN/MANAGER
/inventory	Goods receiving, adjustments, movement history	ADMIN/MANAGER actions
/suppliers, /branches, /notifications	Reference data and alerts	Any authenticated user (writes by role)
/reports, /audit	Sales report and audit history	ADMIN/MANAGER only
/users	User management	ADMIN only

Navigation items are rendered conditionally from the stored role (Layout.jsx), and the router itself is protected by ProtectedRoute, which redirects users who lack the required role. A cashier therefore cannot reach /users even by typing the URL.

3.3 Responsive behaviour

The layout is defined once in styles.css and adapts through media queries:

- **Desktop:** fixed sidebar and main area, a four-column card grid and a two-column POS (product list and cart).
- **At 980 px and below:** cards collapse to two columns, the POS grid becomes one column and the shell stacks vertically.
- **Mobile:** the sidebar becomes a horizontal top bar, cards become a single column and forms stack. The same components are reused; there is no separate mobile code base.

3.4 Wireframes / mockups

The wireframes in Figure 4 describe the three target sizes. They are abstract (boxes and labels) on purpose: they show the layout rules that the implemented screens follow, and they remain valid when visual details change.

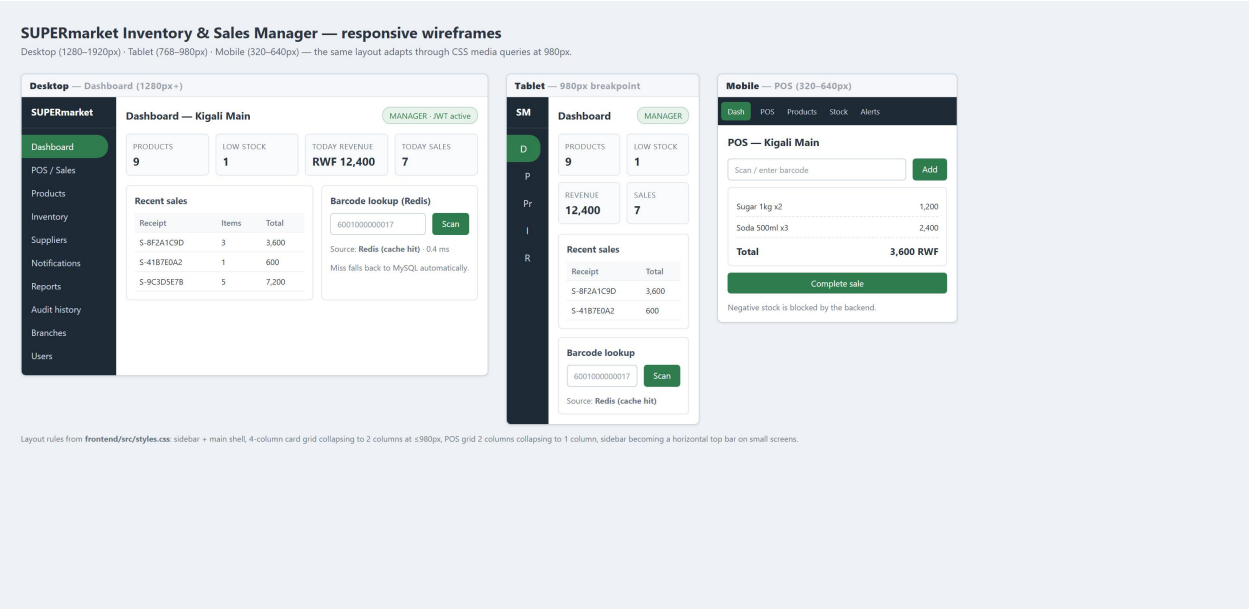


Figure 4: Responsive wireframes - desktop, tablet (980 px breakpoint) and mobile POS.

4. Backend architecture

4.1 Architecture style

The backend is a layered (n-tier) monolith that exposes a stateless REST API. Every request passes through the same chain: the Spring Security filter chain, a controller, a service and a repository. Cross-cutting concerns (validation, error mapping, auditing and caching) are handled by dedicated components instead of being duplicated inside the controllers.

Architecture styles used. The assignment lists monolithic, layered, microservice and event-driven styles. This implementation deliberately combines two of them. The backend is a layered monolith, with web, service and persistence layers inside one Spring Boot unit. The notification side is event-driven: services publish domain events to RabbitMQ and asynchronous consumers react to them (section 7). The frontend and the backend communicate exclusively through the REST API of section 4.3, which returns JSON.

Figure 5 shows the physical deployment as a UML deployment diagram. The user's browser (client device) loads the React single-page application from the Vite server on port 5173, which also proxies /api calls, and sends its JSON requests with a Bearer JWT to the Spring Boot backend on port 8080. The backend is the only component that talks to the data stores and the broker: MySQL over JDBC (3306), MongoDB (27017), Redis (6379) and RabbitMQ over AMQP (5672). All of them run as plain local services on one host, with no containers.

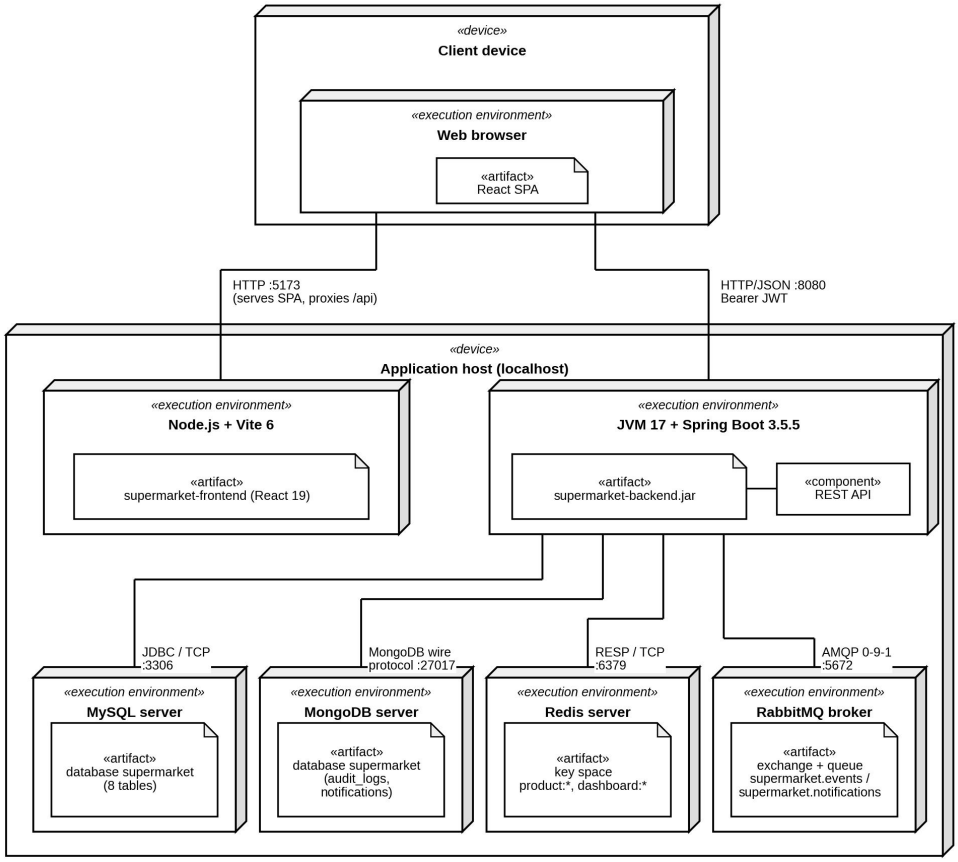


Figure 5: UML deployment diagram - nodes, artifacts and communication paths of the React frontend, Spring Boot backend, MySQL, MongoDB, Redis and RabbitMQ.

4.2 Layers and components

Layer	Package	Responsibility
Web	com.supermarket.web (12 controllers)	HTTP mapping, @Valid request validation, @PreAuthorize role checks
DTO	com.supermarket.dto	Request/response records with Bean Validation annotations
Service	com.supermarket.service (12 services)	Business rules, transactions, cache invalidation, audit and event publication
Security	com.supermarket.security, com.supermarket.config	JWT issuing and verification, AuthContext with branch scoping, security filter chain
Messaging	com.supermarket.mq	DomainEvent, EventPublisher, NotificationConsumer
Persistence	com.supermarket.repository (10 Spring Data repositories)	JPA repositories for MySQL, MongoDB repositories for audit and notifications
Domain	com.supermarket.domain	8 JPA entities plus the Role and MovementType enums
Exception	com.supermarket.exception	Typed exceptions and GlobalExceptionHandler (JSON errors)
Operations	Actuator	Health check used by CI and manual verification

4.3 REST API

The base URL is http://localhost:8080/api. Only POST /api/auth/login and the actuator health endpoints are public; everything else requires a Bearer token. The full list is in docs/api.md and the main endpoints are:

Method	Path	Access
POST	/auth/login	public
GET	/auth/me	authenticated
GET/POST/PUT/DELETE	/products[/{id}]	read: authenticated; write: ADMIN, MANAGER
GET	/products/barcode/{barcode} (and /benchmark)	authenticated / ADMIN, MANAGER
POST/GET	/sales	authenticated (branch-scoped)
POST	/inventory/goods-receiving, /inventory/adjustments	ADMIN, MANAGER
GET	/inventory/movements	authenticated
GET	/dashboard, /notifications, /notifications/unread-count	authenticated
GET	/reports/sales, /audit	ADMIN, MANAGER
GET/POST/PUT	/users	ADMIN
GET/POST/PUT	/branches, /categories, /suppliers	read: authenticated; write: by role
GET	/actuator/health	public

4.4 Validation

- **Declaration:** DTO records carry @NotBlank, @NotNull, @Email, @Min and @DecimalMin constraints.
- **Enforcement:** @Valid on controller parameters triggers Spring MVC validation; invalid bodies return 400 with a fieldErrors array.
- **Business rules:** cross-field rules that Bean Validation cannot express (stock level, branch ownership, duplicate email, deactivated product) live in the services and raise BusinessRuleException.

4.5 Error handling

A single @RestControllerAdvice (GlobalExceptionHandler) maps exceptions to one consistent JSON shape: timestamp, status, error, message and path, plus fieldErrors for validation problems. Typed exceptions cover the common cases: ResourceNotFound (404), DuplicateResource (409), BusinessRule (409/400), Unauthorized (401) and Forbidden (403). Unexpected exceptions are logged on the server and answered with a generic 500, so internal details never leak to the client.

4.6 Sale processing interaction (sequence diagram)

The sequence diagram in Figure 6 shows how POST /api/sales is processed by SaleService.create. Lifelines are written as instance : Class. All product lines are loaded and validated before any change is made. An inactive product, a product of another branch or a quantity above the available stock ends the interaction with a BusinessRuleException and nothing is modified (alt fragment).

Otherwise the sale, its items and one SALE stock movement per product are saved in a single transaction, the cache entries are evicted, the audit entry is recorded and the sale.completed event is published. Finally LowStockNotifier checks the threshold of every product and publishes a stock.low event where needed (loop and opt fragments).

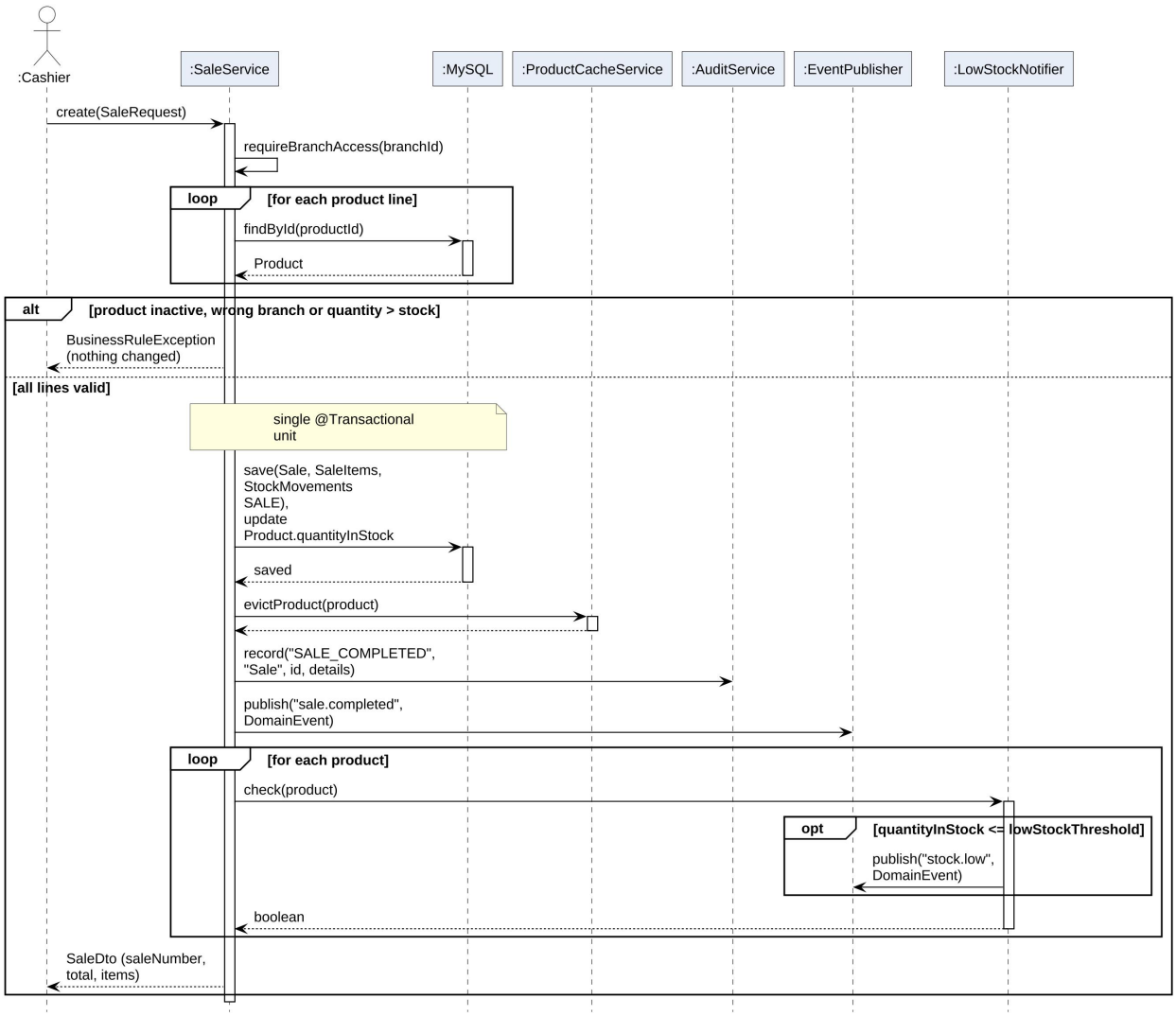


Figure 6: UML sequence diagram - recording a POS sale (SaleService.create).

5. Databases (MySQL and MongoDB)

5.1 MySQL - relational database (source of truth)

- **Purpose:** all operational business data that must be consistent and relational: users, branches, categories, suppliers, products, sales, sale_items and stock_movements.
- **Access:** Spring Data JPA / Hibernate and MySQL Connector/J, database supermarket on localhost:3306 (XAMPP in the laboratory).
- **Why relational:** sales and their lines, products and their branch, and movements and their actor are classic relations with referential and uniqueness constraints. The transactions that keep stock correct need ACID guarantees.
- **Structure:** see section 2.4 - 8 tables, unique barcode and sale number, and indexes on the columns used by queries (barcode, branch, creation date, product).

5.2 MongoDB - non-relational database (documents)

- **Purpose:** append-oriented, schema-flexible data: audit_logs and notifications.
- **Access:** Spring Data MongoDB, database supermarket on localhost:27017.
- **Why a document store:** audit entries describe many different actions with varying details, and notifications are read-heavy and rarely updated. Neither needs joins, but both benefit from an indexed creation time and simple appending.

- **Structure:** see section 2.5 - two collections with indexed actor, branch and time fields.

5.3 Division of labour

Question	Answered by
What does a product cost and how many units are in stock?	MySQL (products)
What was the total revenue of a branch last week?	MySQL (sales, sale_items)
Who changed the stock and why?	MongoDB (audit_logs) + MySQL (stock_movements)
Which users must see the low-stock alert?	MongoDB (notifications)
Is this barcode already cached?	Redis (product:barcode:*)

6. Security, authentication and authorization

6.1 Authentication

Authentication is performed by POST /api/auth/login with an email and a password. The backend looks up the user, verifies the password against a BCrypt hash with BCryptPasswordEncoder, checks that the account is active and issues an access token. Login, success and failure are written to the audit trail. The frontend stores the token in localStorage and attaches it to every request; a 401 response clears it and redirects to the login page.

6.2 OAuth2 / JWT implementation

The project uses the Spring Security OAuth2 stack in resource-server mode. JwtConfig builds a Nimbus HS256 encoder and decoder from a shared secret, JwtService issues the tokens, and the security filter chain validates the Bearer token of every request. The token claims are sub (email), uid (user id), name, role, branch, issuer, issued-at, expiry (8 hours) and jti.

Honest statement about scope

The implemented OAuth2 component is the resource-server / JWT part: token issuing, validation and its use for authorization. No social-login provider such as "Sign in with Google" is implemented, because the code contains no OAuth2 client or redirect flow. This documentation claims only what exists.

6.3 Password protection

- Passwords are stored as BCrypt hashes (users.password_hash); plaintext never reaches the database.
- Hashes are never returned by the API (UserDto does not include them) and are never cached in Redis.
- User creation and update enforce a minimum password length of 6 characters.
- Demo accounts are seeded only for local development.

6.4 Role-based access control (RBAC)

Roles are carried in the JWT (role claim) and mapped to Spring authorities (ROLE_ADMIN, ROLE_MANAGER, ROLE_CASHIER). Method security is enabled and each write endpoint declares its allowed roles with @PreAuthorize.

Capability	ADMIN	MANAGER	CASHIER
Login, dashboard, notifications, own branch products and sales	Yes	Yes	Yes
POS sale	Yes	Yes	Yes

Capability	ADMIN	MANAGER	CASHIER
Create/update/deactivate products, benchmark endpoint	Yes	Yes	No
Goods receiving and stock adjustment	Yes	Yes	No
Low-stock list, sales reports, audit history	Yes	Yes	No
Writing categories and suppliers	Yes	Yes	No
Users and branches management	Yes	No	No
Access to every branch's data	Yes	Own branch only	Own branch only

6.5 Branch-level authorization

Beyond roles, every data access is scoped to a branch. AuthContext.requireBranchAccess(branchId) lets administrators cross branches and rejects anybody else with a 403 if the requested branch is not theirs. Lists of sales, movements, dashboards and reports silently fall back to the caller's branch, so a cashier of Kigali Main can never read the figures of Huye, even with a valid token.

6.6 Handling of configuration and secrets

- All connection details and secrets are read from environment variables with local defaults (MYSQL_URL/USER/PASSWORD, MONGODB_URI, RABBITMQ_HOST/PORT/USER/PASSWORD, REDIS_HOST/PORT, JWT_SECRET, CORS_ALLOWED_ORIGINS).
- The JWT secret committed in application.yml is a development default; production is expected to override JWT_SECRET. No real production secret is stored in the repository.
- CORS is restricted to the configured frontend origins. Sessions are stateless (no cookie to steal) and CSRF protection is disabled because the API is token-authenticated.
- No password, hash or token is ever written to Redis or to the logs.

7. RabbitMQ messaging

7.1 Broker architecture

RabbitMQ is the asynchronous backbone of the application. The backend publishes domain events to a durable topic exchange, and a listener consumes them to produce user-facing notifications.

Element	Value in the implementation
Exchange	supermarket.events (topic, durable)
Queue	supermarket.notifications (durable)
Binding	queue bound to # (all routing keys)
Serialization	Jackson JSON message converter
Connection	localhost:5672, credentials from environment variables
Failure policy	default-requeue-rejected: false (no poison-message loop)

7.2 Events (producers)

Routing key	Event type	Published by	Meaning
sale.completed	SALE_COMPLETED	SaleService	A sale was recorded (items and total)
stock.low	LOW_STOCK	LowStockNotifier	Stock reached the low-stock threshold

Routing key	Event type	Published by	Meaning
stock.received	GOODS_RECEIVED	InventoryService	A supplier delivery added stock
stock.adjusted	STOCK_ADJUSTED	InventoryService	A manual correction with a reason

Each event is a DomainEvent record with a type, a branchId, an optional actorId (target user), a title and a message. Events are published from the service method after the database writes have been issued. The broker call is not part of the database transaction, so in the rare case that the transaction fails at commit after the event was published, a notification could describe a change that was not stored. This trade-off is recorded in section 12.

Scope of the channels

The assignment describes the broker as carrying email, SMS and other events. In this implementation RabbitMQ carries domain events (sale completed, goods received, stock adjusted, low stock), which the consumer turns into in-app notifications displayed by the web interface. No email or SMS gateway is connected, so no real message leaves the system. Adding such a channel would only require another consumer on the same exchange; it is recorded as a limitation in section 12 rather than claimed as implemented.

7.3 Consumers and notifications

NotificationConsumer listens on supermarket.notifications with @RabbitListener and calls NotificationService.createFromEvent. That method stores one document per recipient (all active users of the branch) or a single branch-wide document when no specific user is targeted. The frontend then reads /api/notifications and the unread counter.

7.4 Event flow

Figure 7 shows the event flow as a UML sequence diagram. A producer service hands a DomainEvent to EventPublisher, which sends it to the topic exchange; the exchange routes it to the notification queue, from which NotificationConsumer delivers it to NotificationService, which stores the notifications in MongoDB. Later, a branch user fetches them through GET /api/notifications. The second alternative of the alt fragment is the broker-down path described in section 7.5.

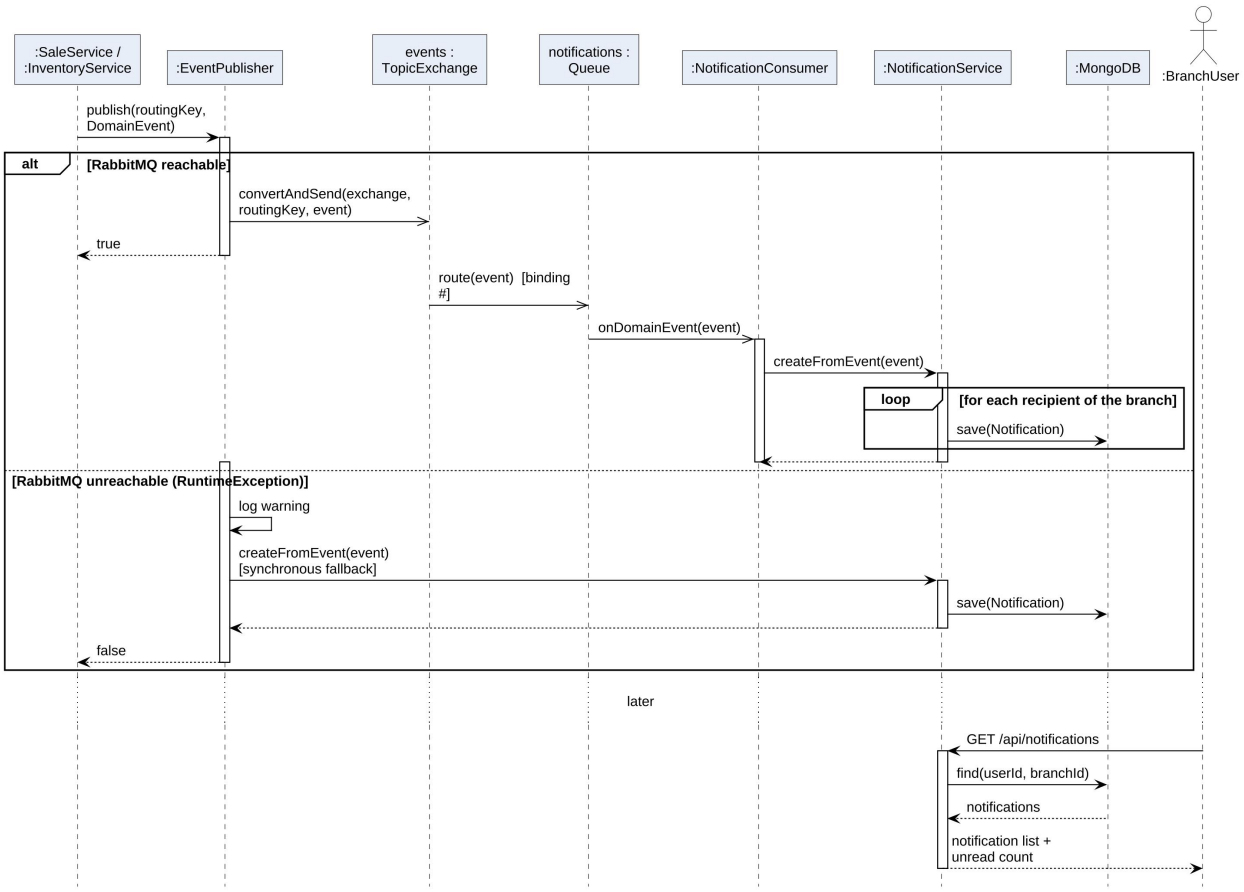


Figure 7: UML sequence diagram - RabbitMQ event flow, from producer to MongoDB notification, including the synchronous fallback.

7.5 Behaviour when the broker is down

EventPublisher tries rabbitTemplate.convertAndSend. If the broker is unreachable, it logs a warning and applies the notification directly through NotificationService (synchronous fallback). The business operation therefore always succeeds: messaging improves decoupling and latency, but it is not a single point of failure. This behaviour is covered by EventPublisherTest.

8. Redis caching (additional technology)

8.1 Why Redis is used

The POS barcode lookup is the hottest read path: every scan repeats the same query for the same few hundred products. Computing the dashboard also aggregates several queries on every page load. Redis answers both from memory instead of re-querying MySQL, which reduces database load and shortens response time. Redis is deliberately a cache only: MySQL stays the source of truth and all cached data is disposable.

8.2 Cache keys and lifetime

Key	Value	TTL	Written when
product:barcode:{barcode}	product JSON	10 minutes	Barcode lookup (miss)
product:id:{id}	product JSON	10 minutes	Product read by id
dashboard:branch:{branchId}	summary JSON	5 minutes	Dashboard request (miss)

Low-stock queries are intentionally never cached: stock correctness always wins over speed.

8.3 Cache behaviour

Figure 8 is a UML activity diagram with swimlanes for the barcode lookup. The POS client sends the request; ProductService asks the Redis cache first. On a hit the response is built directly from the cache and reports cacheHit = true. On a miss, or when Redis is unavailable, the service queries MySQL; an unknown barcode ends with a 404, while a known one is stored in Redis with a TTL and returned with cacheHit = false.

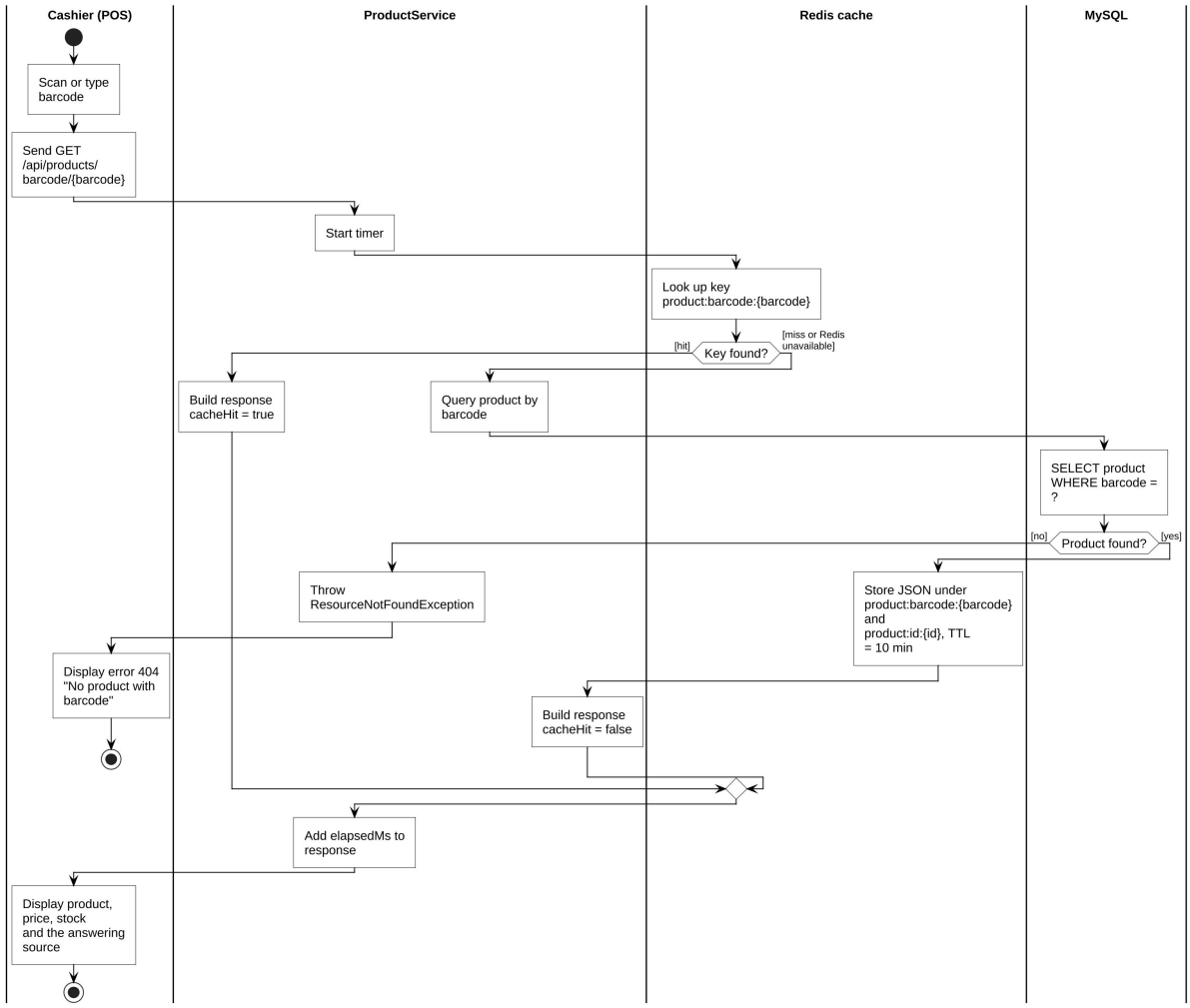


Figure 8: UML activity diagram with swimlanes - barcode lookup with Redis cache hit, miss and failure fallback.

- **Hit:** the cached product is returned, MySQL is not queried, and the response reports cacheHit = true together with the elapsed time.
- **Miss:** MySQL is queried, the result is stored in Redis with a TTL, and cacheHit = false is reported, so the UI can show which source answered.
- **Invalidation:** writes always update MySQL first and then evict the affected keys: product create/update/deactivate, price changes, barcode changes (old and new key), sales, goods receiving and adjustments.
- **Failure:** every Redis command is wrapped; on an outage the code logs one warning and falls back to MySQL, so the request still succeeds (reliability requirement NFR-02).
- **Security:** only product and aggregate JSON is stored, with no passwords, tokens or personal data.

Writes follow the activity diagram in Figure 9. The request is validated first; only a valid and authorised change is persisted, always in MySQL first, and only then are the affected Redis keys evicted. If Redis is unavailable the failure is logged once and the request still succeeds. The next read of the product is a cache miss and reloads fresh data, so stale data is never served.

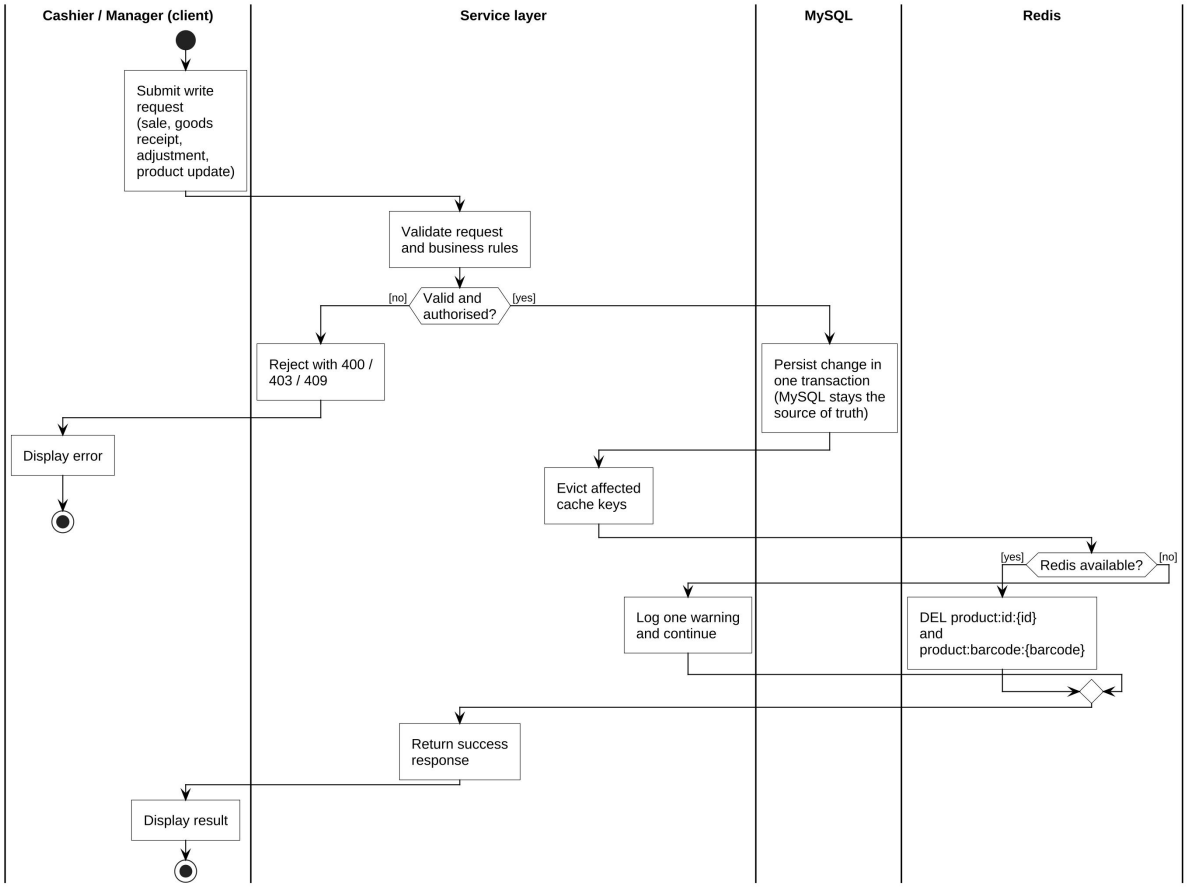


Figure 9: UML activity diagram with swimlanes - write path with cache invalidation.

8.4 Performance evidence

The application exposes GET /api/products/barcode/{barcode}/benchmark?iterations=20 (ADMIN/MANAGER). On the machine running the system it measures the average time of N direct MySQL lookups against N Redis lookups of the same barcode, and returns both averages and the computed speed-up, with a note containing the iteration count and timestamp. The endpoint also reports cacheHit and elapsedMs for each request.

No fixed measurement is quoted in this document: a benchmark number is only meaningful together with the machine and the moment it was taken. The measurement is performed live through the endpoint above during the demonstration, and its output is the evidence of this project.

9. Testing and quality assurance

9.1 Backend tests

Framework: JUnit 5, Mockito, Spring test slices, MockMvc and AssertJ, with H2 in memory for persistence tests (test scope only; production uses MySQL). There are 38 tests in 8 classes, all passing (mvn test, verified locally).

Test class	Tests	What it proves
ApplicationIntegrationTest	6	Full Spring context: login issues a token, the token unlocks barcode lookup, a wrong password fails, protected endpoints return 401, a cashier cannot open an admin endpoint, and dashboard and product list work end to end
ProductApiSecurityTest	7	RBAC over the real security filter chain: anonymous 401, forged token 401, cashier 403 on product creation, manager 200, cashier can scan, benchmark forbidden for cashier, invalid body 400

Test class	Tests	What it proves
ProductServiceTest	8	Redis cache hit (no MySQL query), cache miss (query and store), unknown barcode 404, invalidation on price change and barcode change, duplicate barcode rejected, id cache, benchmark response
SaleServiceTest	4	A sale deducts stock, writes movements and evicts the cache; insufficient stock is rejected with unchanged stock; another branch's sale is forbidden; low stock publishes an event
InventoryServiceTest	5	Goods receiving adds stock and invalidates the cache; an adjustment that would make stock negative is rejected; a valid adjustment applies; a zero adjustment is rejected; another branch's product is forbidden
ProductRepositoryIntegrationTest	4	H2: barcode lookup, unique barcode constraint enforced by the database, low-stock query, deactivated products leave the low-stock list
JwtServiceTest	2	Token round trip: claims (role, branch, expiry) are issued correctly
EventPublisherTest	2	Publish goes to RabbitMQ; when the broker is down the direct fallback still creates the notification

9.2 Frontend tests

Framework: Vitest, React Testing Library and jsdom (npm test). There are 12 tests in 3 files, all passing.

Test file	Tests	What it proves
utils/format.test.js	6	Amount/RWF formatting, null handling, timestamp formatting, cache-hit labels
pages/Login.test.jsx	3	The login form renders, a successful login stores the JWT and navigates, the backend error message is displayed
components/ProtectedRoute.test.jsx	3	Anonymous users are redirected, a wrong role is blocked, a manager is allowed

9.3 Coverage of the required test categories

Category	Covered	Evidence
Unit / business-rule tests	Yes	SaleServiceTest, InventoryServiceTest, ProductServiceTest
Integration / API tests	Yes	ApplicationIntegrationTest (full context), MockMvc suites, repository test on H2
Security / RBAC tests	Yes	ProductApiSecurityTest (401/403/200), branch-scoping assertions
Caching tests	Yes	Hit/miss/invalidation assertions in ProductServiceTest
Messaging tests	Yes	EventPublisherTest, including the broker-down fallback
Frontend tests	Yes	12 Vitest tests (login flow, route guards, formatting)
Automated in CI	Yes	GitHub Actions runs mvn verify and npm test on push and pull request
Load / stress testing	No	No load-test tool is configured; only the benchmark endpoint measures single-lookup timings

10. Git and GitHub workflow

- **Repository:** https://github.com/Emmalise1/Supermarket-Inventory-Sales-Manager (existing repository; main is the primary branch).
- **History:** meaningful commit messages that describe what changed and why; the authorship and timestamps of the original work are preserved.

- **Branching:** work is done on a short-lived branch (this documentation phase used docs/final-documentation) and integrated into main through a pull request.
- **Pull request:** as the assignment requires, the completion of the project is submitted as a pull request targeting the main branch. It is reviewed and merged only with the owner's approval, so main is never rewritten.
- **Review gates:** the CI workflow (section 11) runs on every pull request, so a pull request cannot be merged while the build or the tests fail.

11. DevOps and continuous integration

The pipeline is defined in .github/workflows/ci.yml and runs on every push and pull request targeting main or develop. Two independent jobs run in parallel:

Job	Runner setup	Steps
backend	Ubuntu, Java 17 (Temurin), Maven cache	mvn -B -f backend/pom.xml verify (build and all tests); surefire reports are uploaded as artifacts on failure
frontend	Ubuntu, Node.js 20, npm cache	npm ci, then npm test (Vitest), then npm run build (Vite)

Containers are deliberately not part of this project: the application runs on local services (XAMPP MySQL, MongoDB, RabbitMQ, Redis) and the pipeline needs no container runtime.

Operational verification. GET /actuator/health is public and returns the application status, which is used to confirm that a started instance is alive.

Deployment aspect. The bonus asks for DevOps practices, particularly deployment. The pipeline automates build and test verification for every push and pull request. The application is deployed by starting it with the commands of Appendix A and then confirming it with /actuator/health. Publishing a release automatically to a remote server is not configured, and containers are deliberately excluded from the design, as the project instructions require.

12. Limitations and unimplemented items

The following items were checked against the source code and are stated honestly rather than claimed:

Item	Status in the project
Google / social OAuth2 login	Not implemented. Authentication is local email/password; the OAuth2 resource-server (JWT) part of Spring Security is what is used.
BranchStock, GoodsReceipt, GoodsReceiptItem tables	Do not exist as entities. Stock lives on products (branch_id and quantity_in_stock); a goods receipt is an operation recorded as a StockMovement.
Measured cache benchmark numbers	Not stored in the repository. The benchmark endpoint produces them at runtime; numbers are quoted only from a live run.
Screenshots of the running application	Not produced. The assignment requires diagrams and wireframes, which are provided in Figures 1-9.
Docker	Intentionally excluded from the design and from the CI pipeline.
Load / stress testing	Not configured; only single-request timing through the benchmark endpoint.
Physical barcode hardware, payments, printing	Outside the scope of the web application.
Email / SMS messaging channels	Not implemented. RabbitMQ carries domain events that become in-app notifications; no mail or SMS gateway is integrated.

Item	Status in the project
Event publication timing	Events are published inside the service method, not after the database commit, so a rare commit failure could leave a notification for a change that was not stored. Publishing after commit would be the next improvement.
Automatic deployment to a remote environment	Not configured (bonus item 1). CI builds and tests the application; deployment is a local start verified by the health check, and containers are excluded by instruction.

Appendix A - Demo accounts and running the system

Seeded demo data (only created when the database is empty)

Email	Password	Role	Branch
admin@supermarket.rw	Admin@123	ADMIN	All branches
manager@supermarket.rw	Manager@123	MANAGER	Kigali Main
cashier@supermarket.rw	Cashier@123	CASHIER	Kigali Main

Services required (all local)

- MySQL (XAMPP) on 3306 - database supermarket
- MongoDB on 27017
- RabbitMQ on 5672
- Redis on 6379

Commands

Component	Command
Backend	cd backend && mvn spring-boot:run (port 8080)
Frontend	cd frontend && npm install && npm run dev (port 5173)
Backend tests	cd backend && mvn test
Frontend tests	cd frontend && npm test
Health check	GET http://localhost:8080/actuator/health

Appendix B - Coverage of the assignment requirements

Each numbered requirement of the project instruction is mapped to the part of this document (and of the project) that addresses it. Items that are only partially covered say so explicitly.

#	Assignment requirement	Where it is addressed
1	Problem statement, users, objectives, scope, functional/business requirements, measurable quality attributes, 2-3 user stories with acceptance criteria	Section 1 (1.1 - 1.7): 20 requirements, 6 quality attributes with their measurement, 3 user stories with acceptance criteria
2a	ER-style or UML class diagram: entities, attributes, relationships	Section 2.2 and Figure 2 (UML class diagram); also Figure 1 (use case) and Figure 3 (state machine)
2b	Principal domain concepts (actors, processes, data objects), abstract and implementation-independent	Section 2.1
2c	Technology-oriented schema: primary keys, foreign keys, constraints, indexes	Sections 2.4 (MySQL: PK, FK, unique constraints, indexes) and 2.5 (MongoDB collections and indexes)

#	Assignment requirement	Where it is addressed
3	Responsive pages with a modern front-end framework; mock-ups/wireframes adaptable across devices	Section 3 (React + Vite, media queries) and Figure 4 (desktop / tablet / mobile wireframes)
4	Architecture style(s) with a backend framework, communicating with the front end	Section 4: layered monolith plus event-driven notifications (Figure 5 deployment, Figure 6 sale sequence), REST/JSON API (4.3)
5	Both a relational and a non-relational database	Section 5: MySQL (operational data) and MongoDB (audit and notification documents)
6	Secure authentication including OAuth2, plus performance optimization	Sections 6.1 - 6.3 (OAuth2 resource server / JWT, BCrypt) and Section 8 (Redis cache, Figures 8 and 9, benchmark endpoint)
7	RabbitMQ as message broker for email, SMS and other events	Section 7 and Figure 7: four domain events, topic exchange, consumer, notifications. Email/SMS channels are NOT implemented (sections 7 and 12)
8	RBAC authorization model restricting access to protected resources	Sections 6.4 (role matrix, @PreAuthorize) and 6.5 (branch-level access), verified by security tests (section 9)
9	Git with meaningful commits and clear history; pull/merge request to main on completion	Section 10; pull request #1 (docs/final-documentation -> main) submitted and left unmerged for review
B1	Bonus: DevOps practices, particularly deployment	Section 11: GitHub Actions CI (build, tests, health check). Automatic remote deployment is not configured (stated in section 12)
B2	Bonus: Software testing and quality assurance	Section 9: 38 backend and 12 frontend tests, all passing, plus categories of coverage
B3	Bonus: Incorporate a new technology	Section 8: Redis caching as the additional technology, with the benchmark endpoint

Honest disclosures: email/SMS messaging and automatic remote deployment are not implemented; no fixed benchmark numbers are quoted without a live run; no Google login flow exists - authentication is local password plus an OAuth2/JWT resource server.